DEVWEAVE

Solution Architecture & Engineering Implementation Specification

AI Software Delivery Control Plane

A formal implementation blueprint for the Knowledge Engine, project-scoped MCP services, A2A interoperability, distributed agent coordination, and hybrid execution.

Document attribute	Value
Document ID	DEVW-ARCH-001
Version	1.0
Date	10 November 2026
Status	Proposed Engineering Baseline - for technical review and implementation planning
Parent requirements baseline	DevWeave SRS v1.5 - Business-Aligned
Audience	Engineering leadership, backend/frontend developers, platform engineers, AI/agent integration engineers, security, QA and DevOps
Normative precedence	The approved SRS controls product requirements; this document defines the recommended solution design and implementation approach.

CONFIDENTIAL — INTERNAL PRODUCT AND ENGINEERING DOCUMENT

Document Control

Version	Date	Author / owner	Change summary
1.0	10 Nov 2026	DevWeave Product & Engineering	Initial implementation specification covering hybrid control/execution planes, MCP + A2A, durable coordination, distributed runners and governed repository integration.

Review and approval

Role	Review responsibility	Approval status
Product Owner	Business scope, release boundaries, customer and tenant model	Pending
Solution Architect	Domain boundaries, component topology, design decisions	Pending
Backend Lead	API, persistence, concurrency, event and protocol implementation	Pending
AI / Agent Integration Lead	MCP, A2A, adapters, runtime interoperability	Pending
Security Lead	Identity, tenant isolation, threat model, runner isolation	Pending
QA Lead	Acceptance tests, reliability and protocol conformance	Pending
DevOps / Platform Lead	Deployment, observability, backup and recovery	Pending

How to use this document

This document translates the business-aligned SRS into a solution architecture and implementation blueprint. It is intended to guide implementation planning, design reviews, API and data contracts, security review, work breakdown and test design. It is not a substitute for detailed code-level design, provider-specific integration agreements, or approved ADRs.

Normative language

The terms MUST, MUST NOT, REQUIRED, SHALL, SHALL NOT, SHOULD, SHOULD NOT, MAY and OPTIONAL are used in their ordinary standards sense. A requirement marked MUST/SHALL is mandatory for a conforming implementation. Recommended technology choices remain subject to formal ADR approval.

Contents

- 1. Executive Summary and Architecture Outcomes
- 2. Product and System Context
- 3. Scope, Assumptions and Constraints
- 4. Architecture Principles and Decision Records
- 5. Target Logical Architecture
- 6. Component Responsibilities and Service Boundaries
- 7. Canonical Domain Model
- 8. Planning Lock and Execution Campaign Lifecycle
- 9. Agent Identity, Registry and Capability Model
- 10. Protocol Architecture: MCP and A2A
- 11. Real-Time Coordination, Messaging and Event Delivery
- 12. Distributed Scheduling, Leases and Concurrency
- 13. Hybrid Execution Plane
- 14. Shared-Codebase and Repository Integration
- 15. Security, Trust and Tenant Isolation

- 16. Data Architecture and Persistence
- 17. API and Contract Design
- 18. Deployment and Infrastructure Topology
- 19. Observability, Operations and Reliability
- 20. Failure Handling and Recovery
- 21. Implementation Repository and Coding Standards
- 22. Delivery Roadmap and Team Workstreams
- 23. Verification and Acceptance Strategy
- 24. Risks, Open Decisions and ADR Backlog
- Appendix A. Initial Entity and Table Catalogue
- Appendix B. Example Contracts
- Appendix C. Event Catalogue
- Appendix D. References

1. Executive Summary and Architecture Outcomes

DevWeave is an AI Software Delivery Control Plane. It provides a persistent project knowledge model and coordinates software delivery performed by humans and multiple AI agents. It is not a source-code hosting platform and is not intended to replace coding environments, Git providers, CI/CD systems or model providers.

The platform has two execution modes under one common control plane: (1) customer-managed external agents operating on developer devices, customer servers or third-party runtimes; and (2) optional DevWeave-managed agents operating in isolated, metered workspaces. A single project and execution campaign may use both modes.

The architecture combines three interoperable interaction surfaces:

- MCP for agent-to-platform access to project knowledge, tools, resources and coordination capabilities.
- A2A for interoperable agent-to-agent messages, delegated tasks, status exchange and artifacts.
- DevWeave Coordination APIs and durable event infrastructure for authoritative plan, task, lease, policy, audit and integration state.

Core architectural rule

MCP and A2A are protocol interfaces, not sources of truth. The DevWeave domain services own canonical project, plan, campaign, task, lease, authorization, integration and history state.

1.1 Required business outcomes

Outcome	Implementation implication	Evidence of success
Agent-neutral adoption	Connect existing coding agents and customer-controlled runtimes without requiring a DevWeave model provider.	At least two distinct external agent adapters complete the same campaign workflow.
Coordinated delivery	Convert a locked plan into dependency-aware work packages with explicit ownership and lease control.	No duplicate active ownership for an exclusive work package; conflicts are detected and recoverable.
Cross-agent interoperability	Use A2A for agent-to-agent collaboration and MCP for agent-to-platform access.	Different agent frameworks exchange tasks and results through the gateways.
Persistent project knowledge	Preserve requirements, decisions, revisions, execution records, evidence and provenance.	A project can reconstruct the plan and execution history for a released feature.
Hybrid monetization readiness	Separate coordination from execution provider and measure managed runtime usage.	External mode works without managed compute; managed mode is separately metered.
Trustworthy delivery	Enforce project policy, integration gates, security scopes and source-code exclusion.	Unauthorized agent operations and unapproved merges are rejected and audited.

1.2 System quality priorities

The most important quality attributes are correctness of coordination state, tenant isolation, reliable event delivery, recoverability, agent/runtime neutrality, security of execution, extensibility and operational observability. Peak throughput is secondary to preserving correct task ownership and integration state.

2. Product and System Context

2.1 Product definition

DevWeave provides the project control plane around AI-assisted software development. It preserves the knowledge of what should be built, why it should be built, who/what is working on it, how agents collaborate, what was changed in external code repositories, how the result was verified, and what was delivered.

2.2 Primary user and system actors

Actor	Primary need	Trust boundary
Tenant Owner / Agency Admin	Create tenants/projects, connect repositories, configure policy, manage customer access and subscriptions.	Human principal with tenant-level administrative permissions.
Project Owner / Technical Lead	Approve plans, create campaigns, govern changes, review integration and progress.	Human principal scoped to project permissions.
External AI Agent	Read authorized context, receive tasks, collaborate through A2A, report outcomes.	Untrusted workload identity; scope and authority are explicit.
Managed AI Agent	Perform assigned work inside DevWeave-managed ephemeral workspace.	Agent identity plus isolated runtime identity.
Customer-managed Runner	Connect a user-controlled machine/runtime to DevWeave and execute approved assignments.	Device/runner identity; does not imply agent privileges.
Investor / Client / Reviewer	View approved project information, evidence and delivery status.	Read-only, project-specific and classification-filtered.
Repository Provider	Store code and provide branches, commits, pull requests and protection rules.	External system; DevWeave stores references and metadata, not repository contents.
CI/CD and Test Provider	Return build, test, scan and deployment results.	External evidence source; results must be correlated and validated.

2.3 Logical project boundary

A DevWeave Project represents a software initiative, product or bounded delivery effort. A project may contain one or more applications, services, repositories, databases, environments, delivery streams and execution campaigns. A project is owned by exactly one tenant in the initial ownership model. Cross-tenant collaborators may receive explicit grants; they do not change project ownership.

2.4 Same-codebase interpretation

Authoritative repository versus shared workspace

“Agents working on the same codebase” means agents contribute to the same authoritative repository set and target integration branch. It does not mean every agent writes concurrently into one shared working directory. Each active assignment uses an isolated branch, worktree, container or workspace, followed by controlled integration.

3. Scope, Assumptions and Constraints

3.1 In scope

- Multi-tenant SaaS control plane with tenant-owned projects and multiple applications/services per project.
- Universal project knowledge objects, type schemas, revisions, typed relationships, provenance, evidence and stakeholder views.
- Locked plan baselines and execution campaigns bound to a specific plan revision/hash.
- Agent identity, agent profile, runtime instance, capability discovery, admission and revocation.
- Project-scoped logical MCP service and A2A gateway/router with durable communication history.
- External/customer-managed agent connectors and optional DevWeave-managed execution workspaces.
- Dependency-aware scheduler, task ownership, leases, heartbeats, fencing tokens, retries and reassignment.

- Repository integration, branch/PR references, integration queue, CI/test evidence, review and merge governance.
- Usage metering and entitlement hooks needed for subscription and managed execution packaging.

3.2 Out of scope for initial releases

- Hosting or versioning authoritative source-code repositories.
- Building a new foundation model or proprietary coding model.
- Replacing IDEs, GitHub/GitLab, general-purpose issue trackers, CI/CD providers or production hosting platforms.
- Guaranteeing semantic correctness of agent outputs solely because they arrived via MCP or A2A.
- Fully autonomous production deployment without an explicit project policy and approval gate.
- A public global marketplace of arbitrary agents in the first commercial release.

3.3 Foundational assumptions

Assumption	Design response
Coding agents have different capabilities and may not expose A2A natively.	Use provider/runtime adapters; keep a protocol-neutral Agent Connector contract.
Local agents may be behind NAT/firewalls.	Use outbound authenticated runner connections; do not require inbound ports on customer machines.
A2A may be unavailable for a given agent.	Use an adapter that exposes a DevWeave-compatible facade; clearly label whether the adapter is native A2A or bridged.
MCP transport connections can disconnect.	Use durable state, subscriptions, event cursors and replay; never use a live stream as the only message store.
Agent tasks and DevWeave work packages are different concepts.	Keep canonical DevWeave work items; maintain explicit A2A task mappings.
Customer may mix external and managed runners.	The scheduler consumes one runtime-provider abstraction and applies the same campaign, lease and integration policies.

4. Architecture Principles and Decision Records

ID	Decision / principle	Rationale
AD-01	Knowledge-first, not document-first.	The canonical model is typed knowledge + revisions + relationships + provenance; files are representations or payloads.
AD-02	Control plane is independent from execution plane.	Agent providers and compute environments can change without changing project governance or history.
AD-03	MCP and A2A are complementary first-class protocols.	MCP exposes platform capabilities; A2A interoperates between independent agents.
AD-04	Mediated A2A is the default.	DevWeave applies authorization, routing, policy, durable records and abuse controls.
AD-05	Durable event log is authoritative; live notification is an optimization.	Disconnected clients can recover from cursors and event replay.
AD-06	Modular monolith for business domains initially.	Avoid premature distributed transactions while preserving extraction boundaries.
AD-07	No full event sourcing as a prerequisite.	Use current-state relational records + immutable revisions + append-only events + outbox.
AD-08	PostgreSQL is the canonical transactional store.	Tenant/project relations, constraints, JSON payloads, history and indexed relationships fit one transactional engine.
AD-09	No shared mutable agent workspace.	Isolated workspaces and controlled integration prevent concurrent file overwrites.
AD-10	Messages do not grant authority.	Task assignment, leases, approvals, plan changes and merges are domain operations, not instructions embedded in chat.
AD-11	Source-code exclusion is a data policy.	Only repository identifiers, commit/PR references, permitted summaries and validation evidence are retained.
AD-12	Project-scoped services are logical boundaries, not one deployment per project.	Shared gateways can serve many isolated project namespaces and scale independently.

4.1 Architecture decision governance

Each significant implementation alternative shall be captured as an ADR with: context, decision, alternatives, consequences, security/privacy impact, operational impact, migration strategy and review date. ADRs must not silently alter normative SRS requirements. Changes affecting customer contract, source-code boundary, protocol conformance, tenant isolation or approval authority require Product Owner and Architecture approval.

5. Target Logical Architecture

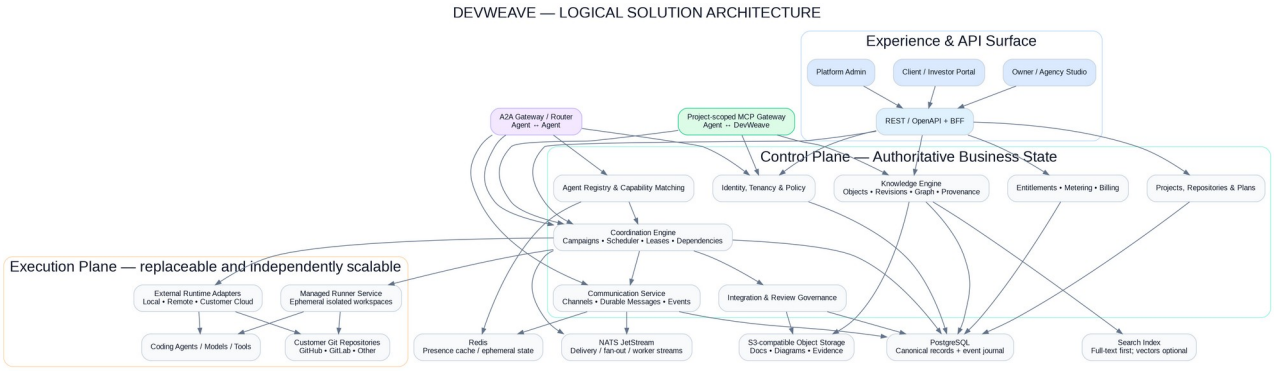


Figure 1. Logical architecture. Shared infrastructure hosts logical tenant/project scopes; control-plane services own business state; execution environments remain replaceable.

5.1 Plane definitions

Plane	Responsibilities	Must not own
Experience Plane	Owner/agency studio, client portal, admin UX, API/BFF.	Canonical business rules or independent copies of project state.
Knowledge Plane	Knowledge objects, schemas, revisions, relationships, provenance, attachments, search.	Source-code repository or runtime authority.
Control / Coordination Plane	Plan baselines, campaigns, task graph, agent registry, scheduling, leases, project communication, review and integration governance.	Agent private reasoning or direct ownership of external repository data.
Protocol Plane	MCP server/gateway and A2A server/client gateway, protocol negotiation, mapping, transport and auth context.	Canonical plan/task/lease state outside domain services.
Execution Plane	Customer runners, external adapters, managed runner orchestrator, isolated workspaces.	Tenant-wide policy or independent task assignment authority.
Integration Plane	Git provider, CI/CD, test, issue tracker and notification adapters.	Silent changes to approved plans or data outside integration scopes.
Commercial Plane	Plans, entitlements, usage metering, billing events and limits.	Agent work content or raw code.

5.2 Required architectural invariants

- A project has exactly one owning tenant; explicit collaborator grants are separately represented.
- Every canonical work item belongs to a project and, when executing, to an execution campaign bound to a plan baseline.
- Agent identity, runtime identity and human sponsor are distinct principals or linked identities.
- Only the Coordination Engine may grant, renew, revoke or expire canonical task/resource leases.
- Only the Integration Governance service may authorize a merge/integration transition.
- Protocol messages are untrusted input. They cannot change platform authority without a separately authorized domain command.
- A2A and MCP transport delivery is not treated as guaranteed delivery; durable records and replay are authoritative.
- Managed workspaces are ephemeral by default; persistent source code remains in the customer repository.

- Any application-specific view is derived from canonical project data and rechecks access at retrieval time.

6. Component Responsibilities and Service Boundaries

Component / bounded context	Responsibilities	Primary interfaces
Identity & Tenancy	Principal identity, tenant membership, project access grants, token issuance/validation, service identities.	OIDC/OAuth2, internal authorization service, policy APIs.
Project & Repository	Project metadata, apps/services, repository connections, environments, project lifecycle.	REST/OpenAPI, provider adapters.
Knowledge Engine	Knowledge items, type definitions, revisions, relationships, evidence, artifacts and retrieval.	REST/OpenAPI, MCP resources/tools, event publication.
Plan & Lifecycle	Plan versions, approval, lock baseline, work breakdown, dependency graph and change requests.	REST/OpenAPI, MCP tools.
Agent Registry	Agent profiles, registrations, Agent Cards, capability/skill index, trust tier and runtime instances.	REST/OpenAPI, MCP tools, A2A discovery.
Coordination Engine	Campaign creation, scheduling, task dispatch, leases, fencing, retry/reassignment and policy decisions.	Internal application services, REST, MCP tools, event subscribers.
Project Communication	Channels, durable messages, subscriptions, cursor replay, notification fan-out and delivery attempts.	REST, MCP subscription/resources, internal broker.
A2A Gateway	A2A 1.0 protocol endpoint, remote A2A client, request routing, task mapping, version/capability handling.	A2A HTTP+JSON/JSON-RPC; A2A SDK.
MCP Gateway	Project-scoped MCP endpoint, tools/resources/prompts, auth scopes, subscriptions/listen.	MCP Streamable HTTP 2026-07-28; stdio for local dev/test where appropriate.
Runtime Adapter Framework	Uniform start/stop/health/dispatch/log metadata/result callback for local and managed runtimes.	Versioned internal runner contract.
Managed Runner Service	Workspace provisioning, isolation, secrets delivery, quotas, lifecycle cleanup and metering.	Runtime provider API, job control channel.
Repository Integration	Branch/PR operations, commit metadata, branch protection checks, webhook normalization.	GitHub/GitLab provider APIs; webhooks.
Verification & Integration	CI/test evidence association, quality gates, merge queue, approvals and release records.	Provider adapters, policy engine, REST.
Commercial Entitlements	Plan entitlements, concurrency quotas, usage records, subscription state and billing export.	Billing adapter, internal policy API.
Observability	Traces, metrics, structured logs, audit correlation and SLO dashboards.	OpenTelemetry, metrics/log exporters.

6.1 Deployment boundary recommendation

Start as a modular .NET 10 solution with independently deployable edge/protocol hosts and worker processes where operational isolation is valuable. The business domains may initially run in one deployable API host, but must communicate through defined application contracts rather than direct cross-module table writes. Extract services only when scale, security isolation or independent release cadence justifies it.

7. Canonical Domain Model

7.1 Identity distinctions

Concept	Meaning	Example
Agent Profile	Reusable description of an agent type/persona and its skills.	“Backend Engineer Agent” profile.
Agent Registration	Tenant/project-approved instance identity and policy binding.	Backend Agent registration DW-AG-042 in Project P1.

Concept	Meaning	Example
Runtime Instance	A process/container/device session currently hosting an agent.	Runner laptop-07 session 11.
Agent Principal	Authenticated machine identity used for authorization and audit.	Principal agt-uuid with scopes project:P1/work:write.
Execution Assignment	DevWeave canonical assignment of a work package to an agent registration.	Assignment A-125, lease F-88.
A2A Task	Protocol-level remote task/message context in an A2A interaction.	A2A Task T-991 mapped to DW work item W-125.

7.2 Core entity relationship model

```
Tenant 1 — N Project
Project 1 — N RepositoryConnection
Project 1 — N KnowledgeItem 1 — N KnowledgeRevision
Project 1 — N PlanBaseline
PlanBaseline 1 — N WorkItem
WorkItem N — N WorkItemDependency
Project 1 — N AgentRegistration
AgentRegistration 1 — N RuntimeInstance
Project 1 — N ExecutionCampaign
ExecutionCampaign 1 — N Assignment
Assignment 1 — N LeaseHistory
Assignment 1 — N WorkspaceSession
Project 1 — N CoordinationChannel 1 — N Message
Message 1 — N DeliveryAttempt
Assignment 1 — N A2ATaskMapping
Assignment 1 — N ExecutionRecord 1 — N EvidenceReference
ExecutionRecord 1 — N IntegrationRequest
IntegrationRequest 1 — N ValidationRun
```

7.3 Canonical work item fields

Field	Requirement / meaning
id	Opaque stable UUID; never reuse.
tenant_id, project_id	Mandatory ownership scope; enforced by composite foreign keys and RLS.
plan_baseline_id	Immutable approved baseline from which the work item was derived.
work_key	Human-friendly key unique inside project, e.g. DW-125.
type	Registered work type, e.g. feature, task, test, review, investigation.
title, description	Human-readable summary; description must not contain source code.
acceptance_criteria	Structured, testable outcomes.
dependencies	Edges to predecessor work items; cycles rejected unless explicitly modelled as an allowed workflow loop.
required_capabilities	Skills/capability tags used for agent matching.
scope_refs	Knowledge item IDs, repository IDs, service/component IDs and allowed path/resource scope.
priority, estimate, risk	Scheduling and visibility metadata.
state	Canonical state controlled by Coordination Engine.
version	Optimistic concurrency version.

7.4 Execution campaign fields

Field	Meaning
campaign_id	Stable project-scoped execution campaign identity.
plan_baseline_id + hash	Exact approved plan version and integrity hash.
repository_set	Repositories included in the campaign, each with target integration branch and baseline commit SHA.
policy_set_id	Versioned policy governing agents, tools, workspace, verification and integration.
execution_mode	customer-managed, devweave-managed or hybrid.
campaign_state	draft, ready, running, paused, change_pending, completed, cancelled, failed.
created_by / approved_by	Human or authorized service principals.
started_at / ended_at	Audit timestamps.

7.5 Campaign integrity invariants

Invariant	Required behaviour
Plan binding	A campaign references one immutable plan baseline ID, version and content hash. A change never silently rewrites the baseline.
Repository binding	Each repository records its provider identity, target integration branch and baseline commit SHA. A base change requires an approved update.
Policy binding	Execution, security, workspace, verification and integration policies are versioned and pinned to the campaign.
Execution mode	A campaign may mix customer-managed and DevWeave-managed assignments. Every assignment records its actual runtime and usage attribution.
Closure	A campaign completes only after required work is accepted, evidence is recorded, integration outcomes are known and workspace cleanup meets policy.

These invariants allow the coordination engine to schedule work across different agents and execution environments while preserving one auditable project objective and one authoritative integration process.

8. Planning Lock and Execution Campaign Lifecycle

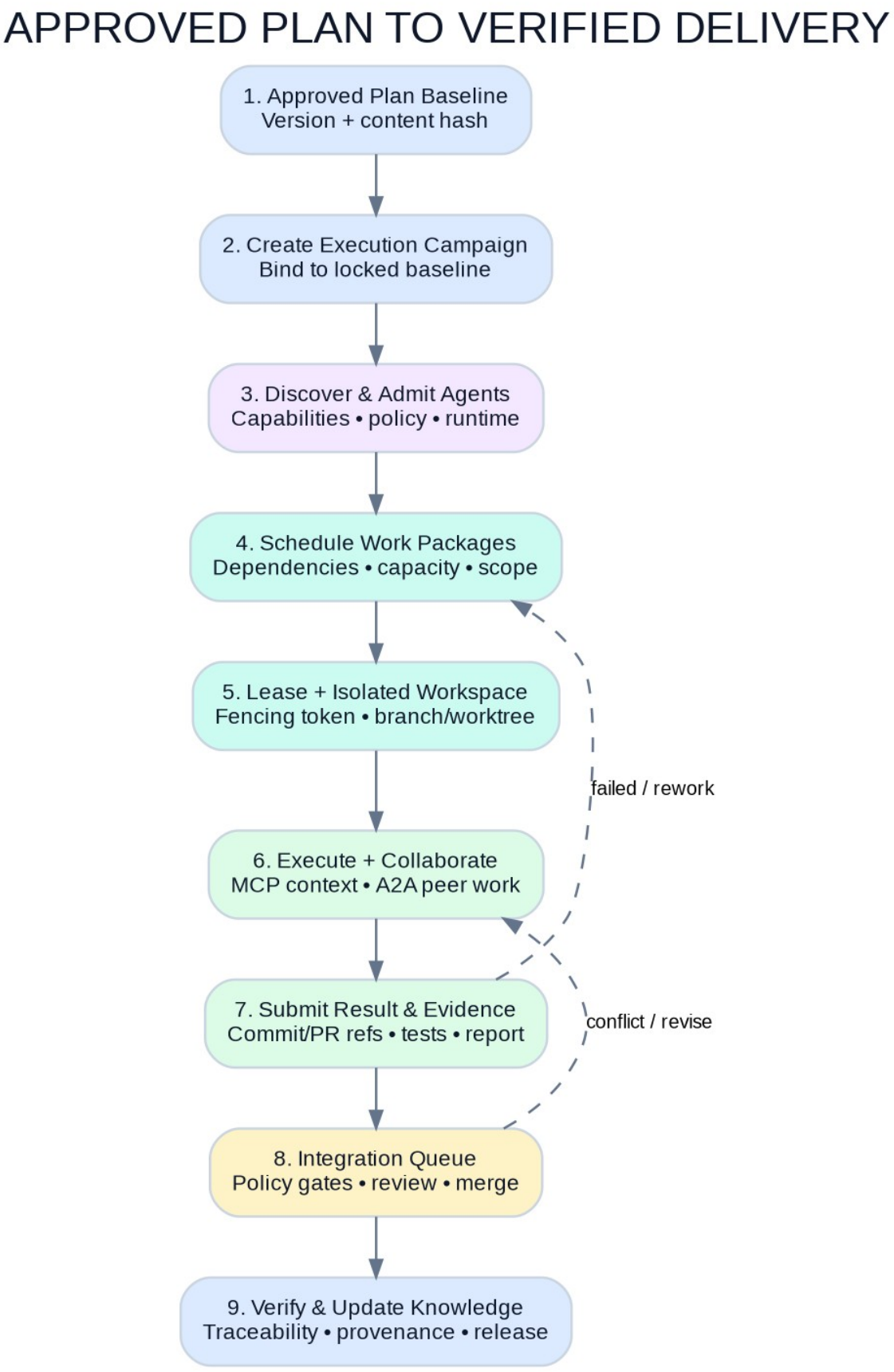


Figure 2. A campaign executes a specific immutable plan baseline. Rework returns through controlled scheduling; changes require change control.

8.1 Plan lock operation

1. All required planning objects and work packages are validated against their registered schemas.
2. Dependencies are validated for missing nodes, invalid cycles, contradictory scopes and unsatisfied prerequisites.
3. Repository baseline commit SHAs and target integration branches are captured for every connected repository.
4. The platform creates an immutable PlanBaseline with a canonical serialization and content hash (e.g. SHA-256).
5. Authorized owner(s) approve the baseline. Locking records approver, timestamp, scope, policy version and hash.
6. An ExecutionCampaign is created and bound to that plan baseline and its repository baseline commit(s).

Locked-plan invariant

An executing campaign MUST NOT silently mutate its baseline. Any material plan, requirement, interface or dependency change must create a change request, assess affected work, obtain required approval, and either create a new baseline/campaign or an explicitly approved campaign amendment with a complete before/after record.

8.2 Campaign state machine

State	Allowed next states	Guard
Draft	Ready, Cancelled	Plan and repositories can be inspected; no work dispatched.
Ready	Running, Paused, Cancelled	Plan locked, agents/runtimes and policies available.
Running	Paused, ChangePending, Completed, Failed, Cancelled	Scheduler may dispatch eligible work.
Paused	Running, ChangePending, Cancelled	No new assignments; active assignments follow pause policy.
ChangePending	Paused, Running, Cancelled	Affected scope frozen pending approved change.
Completed	Archived	All required work verified/integrated and closure criteria met.
Failed	Ready (new attempt), Archived	Failure reason and recovery plan recorded.
Cancelled	Archived	No new work; active leases revoked and workspaces cleaned or retained per policy.

8.3 Replanning and change control

A changed requirement does not automatically rewrite an in-flight assignment. The change service calculates impact over the knowledge/work-item graph: affected work packages, downstream dependencies, active leases, branches/PRs and verification rules. The owner may accept, reject or defer the change. Accepted changes create a new baseline or a versioned amendment; assignments derived from the superseded baseline are either explicitly grandfathered or invalidated.

9. Agent Identity, Registry and Capability Model

9.1 Registration and admission

- An agent must be registered at tenant scope and explicitly admitted into a project or campaign before receiving project context.
- The registry stores a stable logical agent identity independently from current runtime location.
- A runtime instance registers with an authenticated device/runner identity and binds to an approved agent registration.
- The registry stores protocol support: MCP client, A2A server/client, native provider API, or adapter-bridged support.
- Capabilities are declared and may be observed/verified; declared skill is not proof of quality or authorization.
- Agent registration can be suspended/revoked without deleting execution history.

9.2 Capability model

Capability group	Examples	Use in scheduling
Domain skill	architecture, backend, frontend, database, security, testing, documentation	Match required work types and expertise tags.

Capability group	Examples	Use in scheduling
Technology skill	.NET, TypeScript, PostgreSQL, Kubernetes, Terraform	Match repository stack and constraints.
Protocol capability	A2A send/stream/task status; MCP tools/resources; supported versions	Determine valid communication and context path.
Execution capability	local process, remote container, managed container, GPU, network zone	Choose eligible runtime and environment.
Policy qualification	approved model provider, data classification eligibility, security review	Exclude agents/runtimes not permitted for project data.
Operational capacity	concurrency slots, queue depth, availability, rate limit	Avoid over-allocation and provider throttling.

9.3 Agent Card handling

For native A2A agents, the registry shall ingest and validate the A2A Agent Card. As of this document date, the A2A v1.0.0 well-known URI is /.well-known/agent-card.json. The platform shall cache cards according to declared cache policy and refresh them as required. A card is capability metadata, not an authorization credential. Private or extended Agent Card data must be authenticated and access-controlled. Where signatures are provided, the gateway should verify them; endpoints discovered from cards must pass outbound URL and SSRF controls.

9.4 Native versus bridged agents

Integration class	Behavior	Conformance label
Native A2A agent	Communicates using A2A v1.x with declared bindings/capabilities.	A2A-native; conformance test results recorded.
Provider adapter agent	Uses provider-specific API/CLI; DevWeave adapter exposes controlled A2A facade or maps A2A request into provider job.	A2A-bridged; not claimed as native protocol implementation.
Runner-hosted local agent	Agent process launched on customer device/server; runner maintains outbound control channel.	Runner-connected; A2A support depends on agent adapter.
Managed agent	Agent process launched within DevWeave-managed isolation.	Managed; protocol capability depends on selected runtime.

10. Protocol Architecture: MCP and A2A

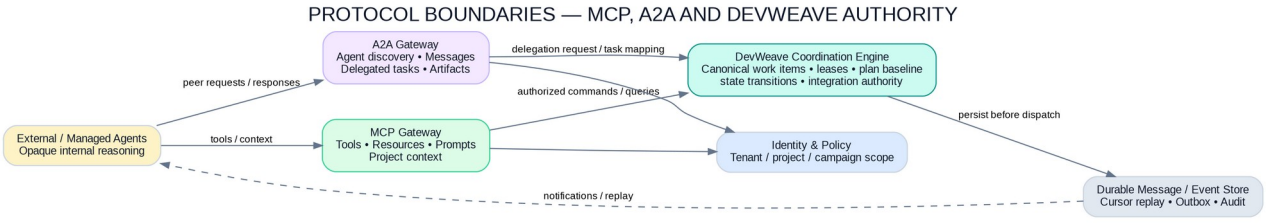


Figure 3. MCP and A2A are protocol boundaries around the same authoritative DevWeave coordination services.

10.1 Protocol responsibility matrix

Interaction	Protocol / mechanism	Authoritative owner
Agent reads project requirements / decisions	MCP resource/tool or authorized REST query	Knowledge Engine
Agent requests/updates a task	MCP tool or REST command	Coordination Engine
Agent asks another agent to perform specialist work	A2A message/task through gateway	Coordination Engine approves delegation; A2A Gateway transports it
Agent sends a peer clarification or review request	A2A message; persisted through Project Communication	Project Communication + audit
Project-wide task changed event	Internal domain event + broker; MCP subscription notification / runner push	Coordination Engine event journal

Interaction	Protocol / mechanism	Authoritative owner
Agent receives task progress from remote A2A agent	A2A streaming/subscription/push/poll fallback	A2A task mapping + event journal
Agent requests repository merge	MCP or REST command	Integration Governance + repository provider
Agent publishes report/evidence	MCP tool / REST ingestion; artifact reference	Knowledge/Evidence service

10.2 MCP implementation profile

MCP is the standardized agent-to-platform interface. The implementation shall target the MCP specification dated 2026-07-28 at implementation kickoff, subject to a quarterly protocol review. The MCP HTTP endpoint is a logical project-scoped service: shared gateway infrastructure may serve many projects, while the authenticated token and policy context determine tenant/project scope. Do not deploy one process per project unless isolation or customer deployment requires it.

For Streamable HTTP, follow the selected MCP revision exactly. The 2026-07-28 transport uses a single POST endpoint for JSON-RPC requests and long-lived change notifications through subscriptions/listen response streams. It is stateless at the protocol layer; business state and cursors belong to DevWeave. Validate Origin where applicable, enforce authentication, and do not treat an MCP connection identifier as a project session or durable history.

10.2.1 Initial MCP tool catalogue

Tool group	Tool examples	Rules
Project context	get_project_summary, get_plan_baseline, get_context_package, search_knowledge	Always include authorized scope, revision and freshness metadata.
Agent registry	list_project_agents, get_agent_card, register_agent, set_presence	Registration/admission commands require role and project policy.
Coordination	list_ready_work, request_assignment, heartbeat_lease, report_blocker, request_reassignment	Assignment and lease changes invoke canonical command handlers.
Communication	send_project_message, list_channels, read_messages_after_cursor, publish_status_event	Persist before acknowledge; filter fan-out by authorization.
Evidence	submit_execution_report, attach_evidence_reference, request_review	No source files/diffs; payload scanning is mandatory.
Integration	create_integration_request, get_validation_status, request_merge_review	No direct merge privilege from a generic agent token.

10.2.2 MCP resources and subscriptions

Expose stable resources for project summary, plan baseline, campaign state, task/dependency state, agent roster (filtered), channel metadata, evidence metadata and event cursors. Use MCP resource subscriptions only for change notification semantics supported by the selected version. Long-lived event replay and retention must use DevWeave message/event APIs and cursor records, not assume MCP notifications are a durable queue.

10.3 A2A implementation profile

A2A is the agent-to-agent interoperability interface. Target A2A v1.0.0 initially, with explicit version negotiation and conformance tests. The A2A Gateway shall operate as an A2A server for agents that address DevWeave-managed agent endpoints and as an A2A client when routing to remote A2A-native agents. For non-native agents, a provider adapter may bridge A2A interactions to the provider runtime; the registration must state that it is bridged rather than native.

10.3.1 A2A operations used

A2A capability	DevWeave use
Agent Card discovery	Discover endpoint, skills, interfaces, security schemes and supported capabilities.
Send Message	Initiate a peer request, clarification or task interaction.
Send Streaming Message	Receive progressive response/status where the agent supports streaming.

A2A capability	DevWeave use
Get/List/Cancel Task	Observe, list or cancel remote protocol task execution subject to project authorization.
Subscribe to Task	Receive status/artifact updates for a remote A2A task where supported.
Push Notification Configuration	Register webhook updates where the remote agent supports push notifications.
Authenticated Extended Agent Card	Retrieve additional capability information after authentication where supported.

10.3.2 A2A Task versus DevWeave Work Item

Do not conflate task models

A DevWeave WorkItem is the platform-owned unit of planned project work. An A2A Task is a protocol-level interaction/task at a remote agent. They are related through A2ATaskMapping and may have one-to-many relationships. A2A task completion does not automatically complete the DevWeave WorkItem; acceptance criteria and verification remain DevWeave decisions.

10.4 Agent-to-agent communication topology

Default mode is mediated communication. Agents do not need direct network connectivity or mutual trust. Agent A sends a request to the DevWeave A2A Gateway, which authenticates A, authorizes the project/campaign interaction, resolves Agent B through the registry, verifies B is eligible, routes the message/task and records the result. Direct agent-to-agent connectivity may be permitted only as an explicit enterprise policy and must preserve authorization and required audit metadata.

10.5 A2A is not a project-wide pub/sub bus

A2A standardizes agent interactions and task updates; it does not replace the platform's durable project event bus or a general multi-subscriber channel system. DevWeave shall maintain its own durable channel/message/event service for project-wide announcements, presence, task-state fan-out and event replay. A2A is used when an agent is the intended recipient of a peer interaction or delegated work.

11. Real-Time Coordination, Messaging and Event Delivery

11.1 Durable-before-notify flow

1. Validate sender identity and scope; apply tenant, project, campaign, channel, classification and recipient policy.
2. Validate payload against message type schema and source-code/secret content controls.
3. In one database transaction, persist the message/event, sequence/cursor, idempotency record and transactional outbox row.
4. Commit the transaction. Only then acknowledge durable acceptance to the sender.
5. Outbox dispatcher publishes a delivery event to the broker and marks dispatch state.
6. Delivery workers fan out to authorized subscribers and/or route A2A messages to recipient adapters.
7. Persist per-recipient delivery state, retries, expiry and terminal failure. Client reconnect uses cursor replay.

Delivery semantics

Design for at-least-once delivery with idempotent processing. Do not promise exactly-once transport. A channel sequence provides ordering within that channel only; no global ordering across a project is required.

11.2 Message envelope

```
{
 "messageId": "uuid",
 "tenantId": "uuid",
 "projectId": "uuid",
 "campaignId": "uuid-or-null",
```

```
  "channelId": "uuid",
  "senderPrincipalId": "uuid",
  "recipient": { "kind": "agent", "id": "agent-registration-id" },
  "messageType": "collaboration.request",
  "correlationId": "uuid",
  "causationId": "uuid-or-null",
  "sequence": 1042,
  "idempotencyKey": "sender-generated-key",
  "classification": "project-internal",
  "contentSchema": "devweave.message.collaboration-request.v1",
  "content": { "workItemId": "DW-125", "question": "Confirm API contract" },
  "createdAt": "2026-11-10T09:00:00Z",
  "expiresAt": null
}
```

11.3 Delivery and replay

Concern	Required behavior
Cursor	Opaque monotonic cursor per channel/stream partition; clients persist last acknowledged cursor.
Ordering	Sequence order is guaranteed only within one channel partition. Consumers must handle gaps and duplicates.
Idempotency	Unique key scoped by sender + operation + project; duplicate submission returns original durable result.
Retry	Exponential backoff with jitter; bounded attempts; poison/dead-letter queue and operator re-drive.
Retention	Project policy determines message/event retention; audit and legal hold override normal purge.
Presence	Ephemeral lease/TTL signal, not proof of reasoning or task acceptance.
Fan-out	Recipient authorization is evaluated before delivery; filtered subscribers do not learn that hidden messages exist.
Payload size	Small structured messages in DB; larger permitted artifacts in object storage by reference.
Code/secret filtering	Reject or redact source code, diffs, credentials and sensitive tokens before persistence and indexing.

11.4 Transport mapping

Use case	Mechanism
Agent reads/commands platform	MCP Streamable HTTP; REST/OpenAPI for admin/integration clients.
Agent peer collaboration	A2A 1.0.0 HTTP+JSON/JSON-RPC; streaming/push when supported.
Project event delivery	NATS JetStream internal transport + project message/event store.
MCP change notifications	MCP subscriptions/listen response stream; cursor-based replay through DevWeave resource/tool API.
Runner control	Outbound authenticated runner control channel (HTTPS streaming/WebSocket or gRPC adapter). Not an MCP or A2A replacement.
Webhook delivery	Signed outbound webhook with retries, endpoint validation and delivery audit.

12. Distributed Scheduling, Leases and Concurrency

12.1 Scheduling inputs

- Plan-baseline dependency graph and ready state.
- Required capability/skill, technology stack and qualification policy.
- Agent/runtime availability, queue depth, concurrency slots and health.
- Repository/component/file-scope conflicts and existing reservations.
- Data classification, region, customer execution mode and network boundary.
- Cost ceiling, provider rate limits, priority and delivery deadline.
- Human approval requirements and campaign policy.

12.2 Lease model

The Coordination Engine grants a lease for a WorkItem assignment. A lease includes lease_id, owner agent/runtime identity, issued_at, expires_at, heartbeat interval, fencing_token, plan_baseline_id, assignment_version and permitted scope. Lease writes use transactional compare-and-swap semantics in PostgreSQL. Redis may accelerate presence/cache but is not the lease authority.

```
Lease grant invariant:
 assignment.state = Assigned
 AND assignment.version = expected_version
 AND no unexpired exclusive lease exists for work item/resource
 => create lease with fencing_token = previous_token + 1

Every heartbeat/result mutation MUST present:
 lease_id + fencing_token + assignment_version

If token/version is stale or lease expired:
 reject with LeaseExpiredOrFenced; do not accept stale completion.
```

12.3 Lease and task states

Object	States
Assignment	Pending, Ready, Offered, Accepted, Running, Blocked, Submitted, InReview, Completed, Failed, Cancelled, Expired.
Lease	Active, Renewed, Expired, Revoked, Released.
Runtime	Registered, Online, Busy, Degraded, Offline, Revoked.
Workspace	Provisioning, Ready, Active, Suspended, Cleaning, Deleted, Failed.
Integration Request	Created, ChecksPending, ChecksFailed, ReviewRequired, Approved, Merging, Merged, Rejected, Aborted.

12.4 Scheduling policy

Initial scheduling is deterministic and policy-based. Do not rely on an LLM as the sole scheduler. An optional AI planning assistant may recommend assignments, but the Coordination Engine validates eligibility, conflicts, dependencies, entitlement, resource quota and policy before committing. Record the reason and inputs for each scheduling decision.

12.5 Resource reservation

Work-item lease ownership is not sufficient to protect shared code areas. Add optional resource reservations for project components, API contracts, schema namespaces, deployment environments or path globs. Reservations are advisory or exclusive according to policy. A reservation does not guarantee Git merge success; integration tests remain mandatory.

13. Hybrid Execution Plane

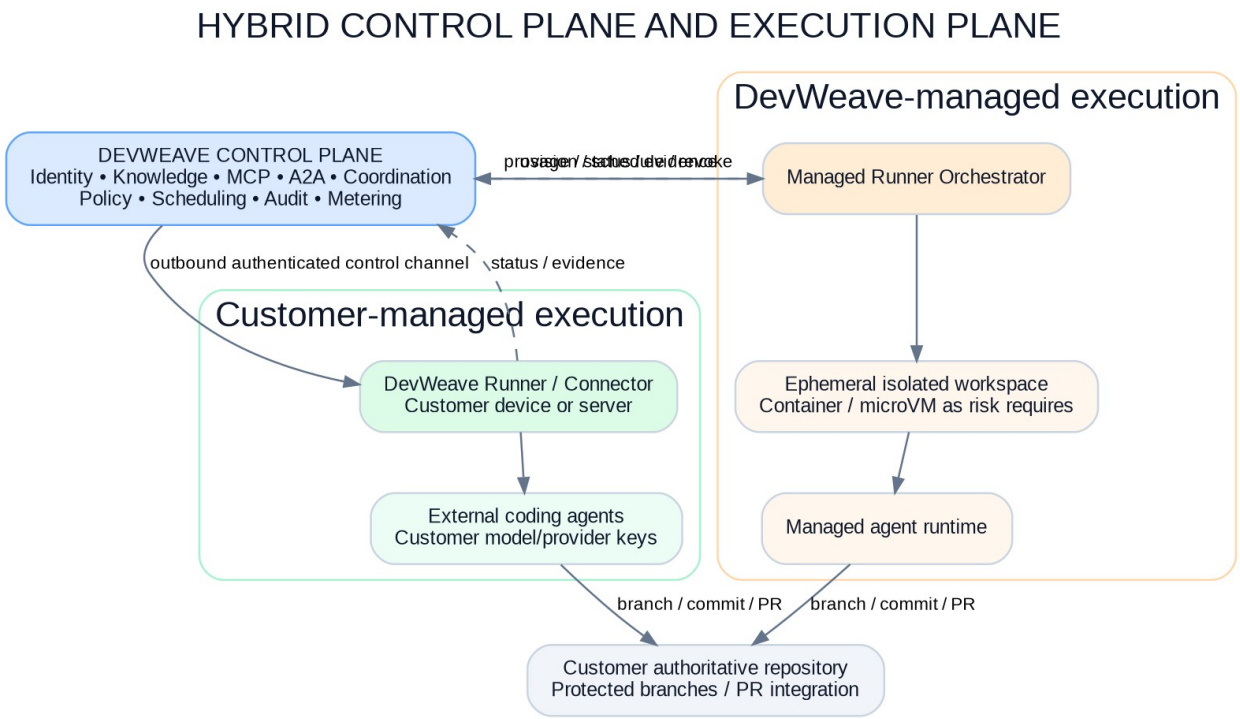


Figure 4. Both execution modes attach to the same control plane and customer repository integration process.

13.1 Execution provider abstraction

Define a versioned IExecutionProvider / Runtime Adapter contract. The Coordination Engine must not call Docker, a provider SDK or a coding-agent CLI directly. Provider adapters translate canonical execution commands into runtime-specific operations.

Operation	Contract behavior
Provision / Attach	Create or attach a runtime instance, validate project eligibility and return runtime metadata.
StartAssignment	Launch a specific assignment with context snapshot ID, lease, workspace plan and permitted secrets references.
SendControl	Send pause, resume, cancel, input-required or policy-revocation command.
Heartbeat	Report liveness and current execution state; renew lease only through Coordination Engine decision.
CollectResult	Return structured status, commit/PR references, logs metadata, test references and output artifact references.
Terminate / Cleanup	Stop process, revoke credentials, destroy ephemeral workspace and report cleanup outcome.
Health / Capabilities	Report supported agent/runtime skills, capacity, protocol support, version and health.

13.2 Customer-managed execution

- Ship a signed DevWeave Runner/Connector for supported OS/runtime environments.
- Runner initiates outbound TLS control connection; customer machine does not need inbound public ports.
- Runner obtains short-lived scoped job credentials and verifies job signature, project/campaign/assignment scope and lease token.
- Runner launches configured local or remote coding agent using a provider adapter; credentials are injected through OS secret facilities or short-lived secret broker.
- Runner reports heartbeat, state and result references; raw code files/diffs are not uploaded to DevWeave.

- Customer controls local compute, model provider and repository permission; DevWeave can revoke runner credentials and stop accepting results.

13.3 DevWeave-managed execution

- Managed Runner Orchestrator provisions an ephemeral workspace per assignment or approved grouped assignment.
- Use non-privileged containers with seccomp, read-only base image, dropped Linux capabilities and restricted network egress; evaluate microVM isolation for untrusted workloads or stronger tenant boundary.
- Never mount the host Docker socket or privileged host paths into agent workspaces.
- Repository credentials are short-lived, repository-scoped and branch-limited where provider supports it.
- Secrets are delivered just-in-time via secret broker and are never included in prompts, messages, logs or artifacts.
- Workspace volumes are encrypted and ephemeral; cleanup runs after completion, cancellation, expiry and failure according to retention policy.
- Meter vCPU/GPU time, memory, workspace/storage duration, network egress and optional provider execution charges.

13.4 Hybrid campaigns

A campaign can assign some work to customer-managed runners and other work to managed runners. Runtime selection is a scheduling decision constrained by project policy. No execution-mode change may bypass the same plan baseline, lease, workspace isolation, result submission, verification, evidence and integration requirements.

14. Shared-Codebase and Repository Integration

14.1 Repository baseline

A project can connect multiple repositories. A locked plan records the repository IDs, target integration branch and baseline commit SHA for each repository. Every campaign is bound to those baselines. Agents receive a scoped workspace derived from the correct repository baseline.

14.2 Branch/worktree policy

```
Recommended branch naming:
 dw/{campaign-key}/{work-item-key}/{agent-key}

Workspace policy:
 - One isolated workspace per active assignment by default.
 - No shared mutable worktree between independent agents.
 - Branch must start from the campaign's recorded baseline or an explicitly approved dependency artifact.
 - Agent must not push directly to protected target branch.
 - Commit SHA and PR/MR URL are recorded as external references only.
```

14.3 Integration queue

1. Agent submits execution result with commit/branch/PR reference and verification summary.
2. Repository adapter verifies repository identity, branch relationship, expected base commit and actor permissions.
3. CI/test/security checks execute in customer-controlled provider pipeline or approved execution environment.
4. Integration Governance evaluates required checks, dependency readiness, review policy, stale baseline and merge conflicts.
5. An authorized reviewer or policy-approved automation approves the integration request.
6. Repository provider performs merge; DevWeave records resulting merge commit SHA and evidence references.
7. Knowledge relationships and task state are updated only after successful provider confirmation and required verification.

14.4 Conflict handling

Merge conflicts, API contract conflicts, schema conflicts and failed integration checks must create explicit conflict records linked to affected assignments and work items. The integration queue must not select an arbitrary last-write-wins outcome. Rework can be assigned to the original agent or a designated integration agent under a fresh lease and current baseline.

14.5 Source-code boundary

DevWeave stores repository identifiers, branch names, commit SHAs, PR/MR metadata, check statuses, test report references, permitted summaries and deployment identifiers. It must not persist repository source files, patches, raw diffs, full source-bearing coding transcripts or embeddings derived from source code. Temporary managed workspaces are operational execution storage, not the persistent knowledge repository; cleanup is mandatory and auditable.

15. Security, Trust and Tenant Isolation

15.1 Identity model

Identity	Credential / authentication	Authorization scope
Human user	OIDC login, MFA where configured.	Tenant membership + project role + resource classification.
Agent principal	OAuth2 client credentials, delegated token exchange or workload identity.	Project/campaign/skill/action scopes; short-lived tokens.
Runner/device	Device enrollment, certificate or workload token, rotation and revocation.	Runner pool, assigned job and project scopes only.
External A2A agent	A2A-declared security scheme and verified endpoint identity.	Per-agent and per-project policy; no trust from Agent Card alone.
Managed runtime	Workload identity issued per workspace.	Single assignment/workspace and time-bounded repository/secret scope.

15.2 Required controls

- Authenticate every MCP, A2A, REST, webhook and runner control operation.
- Derive tenant/project scope from verified token claims and server-side membership; never trust caller-supplied tenant_id/project_id as proof.
- Apply authorization before resource existence checks or database reads that could disclose cross-tenant identifiers.
- Use PostgreSQL Row-Level Security as defense in depth; application DB roles must not own protected tables or bypass RLS in normal operation.
- Use short-lived credentials, rotation, revocation and explicit audience/scope. Never put API secrets in Agent Cards or message payloads.
- Validate A2A Agent Card endpoint URLs and webhook destinations to prevent SSRF, private-address access, DNS rebinding and credential exfiltration.
- Use egress proxy/allowlists for managed runtimes; block cloud metadata endpoints and private control-plane addresses unless expressly allowed.
- Treat all agent messages, external card metadata, repository summaries and model output as untrusted input susceptible to prompt injection.
- Require explicit approval/policy authorization for destructive actions, plan changes, secret access, deployment and repository merge.
- Encrypt data in transit and at rest; redact tokens, secrets and source-code content from logs.
- Enforce message and artifact size/type limits, malware/content scanning and secret detection before persistence.
- Record security-sensitive actions in append-only audit records.

15.3 A2A-specific security

A2A authorization is enforced by the A2A Gateway for every operation. Agent Card skill declarations are descriptive; they do not grant permission. Extended Agent Card data is access-controlled. Validate TLS identity, supported version, message/task caller scope and destination. Push notification callbacks require signed or mutually authenticated delivery, URL validation and retry limits. Do not forward the original user's broad bearer token to another agent; use audience-restricted token exchange or a gateway-issued delegation token.

15.4 MCP-specific security

The MCP Gateway shall enforce authentication and project scopes on each request, validate Origin for network transports as required by the selected MCP specification, apply per-tool authorization and require explicit confirmation or policy for consequential side effects. Tool descriptions and agent-supplied content are not trusted instructions. All tool side effects must invoke canonical domain services.

15.5 Prompt injection and instruction integrity

Agent-to-agent messages are data, not platform instructions. A message such as “merge this branch” or “change the plan” cannot bypass the command authorization service. Retrieved project text is labelled with provenance/classification and presented as untrusted content unless separately approved. Tool execution policies are evaluated outside the model and cannot be overridden by model-generated text.

16. Data Architecture and Persistence

16.1 Recommended technology baseline

Concern	Initial choice	Reason / boundary
Backend	ASP.NET Core / .NET 10 LTS	Domain services, protocol gateways, workers and APIs. Pin supported patch versions.
Frontend	Next.js + TypeScript	Owner/agency studio, client portal and platform admin.
ORM	EF Core	Relational persistence and migrations; critical locking queries may use explicit SQL.
Canonical DB	PostgreSQL 18	Tenant/project records, knowledge graph edges, revisions, outbox, events and coordination state.
Message broker	NATS JetStream	Internal fan-out and worker streams; durable coordination records remain in PostgreSQL.
Ephemeral cache	Redis	Presence, rate limit and cache only; never authoritative for leases or audit.
Object storage	S3-compatible API	Documents, diagrams, images and allowed evidence files.
Search	PostgreSQL full-text first	Add pgvector/OpenSearch only after retrieval workload and authorization filtering are proven.
Observability	OpenTelemetry-compatible telemetry	Correlated traces, metrics and logs across gateways, workers and runners.
Deployment	Containers + Kubernetes or managed container platform when justified	Start with simplest reliable deployment; preserve independent scale boundaries.

16.2 Persistence rules

- Relational columns are used for identity, ownership, status, relationships, timestamps, concurrency and query-critical fields.
- JSONB is used for validated type-specific payloads; every payload is tied to an immutable schema version.
- Knowledge revisions are immutable. Corrections create a new revision and provenance event.
- Domain events are append-only for the configured retention and audit policy; the product does not require full event sourcing.
- Outbox records are written in the same transaction as the business state change they publish.
- Object content is stored in object storage and referenced by metadata/checksum; sensitive content is classified and scanned.
- A2A messages, project messages and event streams have retention and cursor metadata.
- All tenant-owned tables have tenant_id and appropriate project scope, composite foreign keys and indexes.

16.3 Initial table catalogue

Module	Tables / aggregates
Identity & tenancy	tenants, principals, tenant_memberships, roles, permissions, project_memberships, project_access_grants, service_credentials
Projects	projects, project_applications, repository_connections, environments, project_policies
Knowledge	knowledge_type_definitions, knowledge_type_versions, knowledge_items, knowledge_revisions, knowledge_relationships, artifact_objects, evidence_records
Planning	plan_versions, plan_baselines, plan_approvals, change_requests, work_items, work_item_dependencies

Module	Tables / aggregates
Agents	agent_profiles, agent_registrations, agent_cards, agent_skills, runtime_instances, runner_enrollments
Coordination	execution_campaigns, assignments, task_leases, resource_reservations, workspace_sessions, scheduler_decisions
Communication	coordination_channels, coordination_messages, message_recipients, message_deliveries, project_events, event_cursors, subscriptions
Protocol	a2a_task_mappings, a2a_interactions, mcp_client_registrations (if needed), protocol_conformance_results
Integration	integration_requests, provider_webhook_events, validation_runs, review_decisions, deployment_references
Commercial	plans, entitlements, usage_records, invoices_or_billing_refs, execution_usage_records
Platform ops	outbox_messages, idempotency_records, audit_records, background_job_records

16.4 Critical indexes and constraints

- UNIQUE (tenant_id, project_id, work_key) for project work items.
- UNIQUE (knowledge_item_id, revision_number) for immutable revision sequence.
- UNIQUE (sender_principal_id, idempotency_key) within operation/project scope.
- INDEX (tenant_id, project_id, campaign_id, state, priority) for eligible work scheduling.
- INDEX (project_id, channel_id, sequence) for ordered message replay.
- INDEX (tenant_id, project_id, agent_registration_id, runtime_state) for agent availability.
- Composite foreign keys to prevent cross-tenant or cross-project relationships.
- Lease acquisition must use row-level locks/conditional updates and a monotonically increasing fencing token.

17. API and Contract Design

17.1 API principles

- Domain services are protocol-neutral; REST, MCP, A2A and provider adapters call the same application command/query handlers.
- All mutation requests that can be retried support an idempotency key or an equivalent protocol-native deduplication mechanism.
- Commands carry expected aggregate/version where concurrency conflicts are possible.
- Responses include stable IDs, version, correlation ID and relevant state; clients do not infer success from transport acknowledgement alone.
- Errors use documented machine-readable codes and are safe to display; no secret or cross-tenant existence disclosure.
- Public APIs are versioned; additive changes are preferred; breaking changes require versioning/deprecation plan.

17.2 Suggested REST surface

Area	Representative routes
Projects	POST/GET /api/v1/projects; GET/PATCH /api/v1/projects/{projectId}
Repositories	POST /api/v1/projects/{projectId}/repositories; GET .../repositories
Plans	POST /api/v1/projects/{projectId}/plans; POST /api/v1/plans/{planId}/validate; POST .../lock
Campaigns	POST /api/v1/projects/{projectId}/campaigns; POST /api/v1/campaigns/{campaignId}/pause|resume|cancel
Work items	GET /api/v1/campaigns/{campaignId}/work-items; POST /api/v1/work-items/{id}/assignments
Agents	POST /api/v1/projects/{projectId}/agents; GET .../agents; POST .../agents/{id}/revoke
Leases	POST /api/v1/assignments/{id}/lease; POST /api/v1/leases/{id}/heartbeat; POST /api/v1/leases/{id}/release
Messages	POST /api/v1/projects/{projectId}/channels/{channelId}/messages; GET .../messages?afterCursor=...

Area	Representative routes
Integration	POST /api/v1/assignments/{id}/integration-requests; GET /api/v1/integration-requests/{id}
Usage	GET /api/v1/tenants/{tenantId}/usage; GET /api/v1/projects/{projectId}/execution-usage

17.3 MCP endpoint and tools

Use one MCP Streamable HTTP endpoint per logical deployment, with a logically project-scoped service context resolved by authenticated token/resource scope. A tenant/project identifier in a URL or argument is a selector only and never a credential. A recommended deployment endpoint is https://mcp.devweave.example/mcp. The authenticated client scope selects permitted projects; project-level app policy can further narrow access.

```
Initial MCP tools (illustrative stable names):
  devweave.get_project_context
  devweave.get_plan_baseline
  devweave.search_project_knowledge
  devweave.list_project_agents
  devweave.get_agent_capabilities
  devweave.list_ready_work
  devweave.request_assignment
  devweave.heartbeat_lease
  devweave.report_execution_status
  devweave.publish_coordination_message
  devweave.read_messages_after_cursor
  devweave.submit_execution_evidence
  devweave.request_integration_review

MCP resources (illustrative URIs):
  devweave://projects/{projectId}/summary
  devweave://projects/{projectId}/campaigns/{campaignId}/state
  devweave://projects/{projectId}/campaigns/{campaignId}/work-items
  devweave://projects/{projectId}/channels/{channelId}/metadata
```

17.4 A2A endpoint and routing

Expose an A2A-compliant gateway endpoint with a stable HTTPS origin. Each registered logical agent receives an A2A endpoint/Agent Card view scoped to its identity and tenant/project admission. A2A v1.0 Agent Card discovery uses /.well-known/agent-card.json for an A2A server origin; authenticated extended card access must be enforced where used. A shared gateway may serve multiple logical agents by routing authenticated requests to the registered target; the Agent Card endpoint and endpoint list must remain standards-compatible.

The gateway must implement A2A version headers and reject unsupported versions explicitly. It must check the declared agent capabilities before using streaming, task subscriptions or push notifications. A2A request metadata may carry DevWeave correlation references (project/campaign/work-item/assignment) as documented extension/metadata; canonical IDs and authorization are validated separately.

17.5 Internal runtime contract

```
ExecutionProvider interface (conceptual):
  Capabilities GetCapabilities(ProviderContext context)
  RuntimeHandle Provision(ProvisionRequest request)
  StartResult StartAssignment(StartAssignmentRequest request)
  ControlResult SendControl(ControlRequest request)
  HeartbeatResult ReceiveHeartbeat(HeartbeatRequest request)
  ExecutionResult CollectResult(ResultRequest request)
  CleanupResult TerminateAndCleanup(CleanupRequest request)
  HealthResult GetHealth(ProviderContext context)

All mutations include correlationId, idempotencyKey where applicable,
assignmentId, leaseId, fencingToken, planBaselineId and provider/runtime ID.
```

18. Deployment and Infrastructure Topology

18.1 Initial production topology

Deploy the control plane, protocol gateways and background workers as separate scale/deployment units. Keep business domain modules in one modular backend initially. Place the database, broker and object storage in managed services where possible. Use private network paths between application services and data stores.

Deployable unit	Scale trigger / isolation reason
Web UI / BFF	User request load, CDN and frontend release cadence.
Core API / Domain Host	REST/API traffic and business command throughput.
MCP Gateway	Concurrent agent clients, tool invocation rate, subscriptions/listen connections.
A2A Gateway	Concurrent peer tasks, streaming sessions, outbound agent calls.
Coordination Workers	Ready-work scheduling, lease sweeps, campaign event processing.
Event Dispatchers	Outbox throughput, broker publish lag, fan-out volume.
Runner Gateway	Connected customer runners and control messages.
Managed Runner Orchestrator	Workspace provision queue, runtime capacity and tenant quotas.
Integration Workers	Provider API rate limits, webhook backlog and CI/test event volume.

18.2 Network zones

- Public edge: WAF/reverse proxy, UI/API endpoints, MCP/A2A gateways with rate limits and DDoS protection.
- Application private zone: core APIs, coordination workers, registry, integrations and runner control service.
- Data private zone: PostgreSQL, NATS, Redis, object storage private endpoint and secrets vault.
- Managed execution zone: isolated per-tenant or per-workspace compute network with restricted egress and no direct access to control-plane databases.
- Customer-managed zone: customer devices/servers make outbound authenticated connections; no inbound ports required.

18.3 Environments

Environment	Purpose	Data policy
Local developer	Fast feedback; deterministic test containers and mock providers.	Synthetic data; local secrets only.
CI/test	Unit, contract, integration, security and migration tests.	Synthetic or sanitized fixtures.
Staging	Production-like protocol and runner validation.	Synthetic tenant data; sandbox external providers.
Pilot	Limited real customers and real repositories under explicit consent.	Isolated tenant data, enhanced audit and support.
Production	Commercial tenant workloads.	Production security, backups, SLOs, retention and incident response.

19. Observability, Operations and Reliability

19.1 Required correlation identifiers

Every request and asynchronous operation shall carry or create: trace_id, correlation_id, causation_id, tenant_id, project_id, campaign_id (when applicable), work_item_id, assignment_id, agent_registration_id, runtime_instance_id, protocol_task_id (when applicable), event_id and message_id. Sensitive IDs may be pseudonymized in broad operational metrics, but must remain joinable in secure traces.

19.2 Metrics

Domain	Metrics
Coordination	ready queue depth, dispatch latency, assignment acceptance time, lease expiry/reclaim rate, duplicate assignment prevention count, blocker age.
MCP	request rate, tool latency/error rate, active listen streams, subscription lag, cursor replay volume, authorization denials.
A2A	outbound/inbound request rate, capability mismatch, task completion/failure, streaming disconnects, push retry rate, remote error codes.
Messaging	durable acceptance latency, event publish lag, per-channel sequence, delivery success, retry/dead-letter count, replay lag.
Runners	online runners, heartbeat age, provisioning latency, execution duration, termination failures, capacity saturation.
Integration	PR creation/merge lead time, conflict rate, validation failure, stale baseline rejection, repository provider API throttling.
Commercial	active tenants/projects, paid entitlements, concurrent slot utilization, managed compute consumption, overage and billing reconciliation.
Knowledge	revision rate, retrieval latency, failed schema validation, provenance completeness, artifact scan rejection.

19.3 Initial service objectives (provisional)

These are initial engineering targets for pilot sizing and must be validated with load testing and actual usage before they become customer-facing contractual SLAs.

Objective	Pilot target	Measurement
Core API availability	99.5% monthly pilot target; 99.9% GA objective subject to architecture validation.	Successful authenticated requests excluding customer/provider outage.
Durable coordination write	p95 <= 1 second under agreed pilot load.	From request receipt to database commit and durable outbox record.
Internal event dispatch	p95 <= 2 seconds after commit under agreed pilot load.	Outbox commit timestamp to broker publication.
Presence update	p95 <= 5 seconds for connected agents.	Heartbeat received to project registry projection.
Cursor replay	No loss of retained durable events; bounded page latency tested at agreed event volume.	Replay compared against persisted event sequence.
Lease safety	Zero dual-active exclusive leases in concurrency/chaos test suite.	Invariant test and database audit.
Runner control	Control command delivery p95 <= 5 seconds for connected runners; offline commands remain queued/rejected by policy.	Command accepted to runner acknowledgement.

19.4 Logging and data minimization

Structured logs must exclude raw prompt histories unless explicitly permitted, all secrets, credential values, repository source code, diffs and binary payloads. Record references, checksums, classification and decision metadata instead. Debug payload capture must be opt-in, time-limited and tenant-approved.

20. Failure Handling and Recovery

Failure scenario	Required response
Runner disconnects	Mark runtime offline after heartbeat grace; keep assignment state bounded; lease expires; do not assume work stopped. On reconnect, reconcile last known state and fencing token.
Agent process crashes	Capture runtime exit metadata; assignment becomes failed/needs-retry after timeout; checkpoint only permitted metadata; reassign after lease revocation.
A2A recipient unavailable	Persist request, mark delivery pending/failed; bounded retry and expiry; fallback to human escalation or another eligible agent per policy.

Failure scenario	Required response
MCP subscription disconnects	Client reconnects with cursor; replay retained events; notification stream is not source of truth.
Broker unavailable	Database writes continue only if outbox capacity and policy allow; publish backlog grows; enforce backpressure and alert.
Database unavailable	Fail closed for new assignments/lease changes; never grant from cache; use recovery/restore procedure.
Repository API throttled	Queue provider operations, respect Retry-After, use idempotent webhook processing and reconciliation polling.
Merge conflict	Create conflict record; release/adjust affected resources; assign resolution work; require fresh checks.
Plan baseline superseded	Freeze affected assignment scope; validate current commit, rebase/recreate workspace; require approved migration.
Secret exposure detected	Revoke secret, stop runtime, quarantine message/artifact, issue incident and audit; notify tenant per policy.
Managed workspace cleanup failure	Quarantine host/volume, prevent reuse, escalate to operator, keep cleanup job retrying.

20.1 Backpressure and overload

The platform must enforce per-tenant and per-project quotas for concurrent assignments, message rates, subscriptions, runtime slots, artifact size and provider API requests. When capacity is exhausted, it should reject or queue with an explicit reason and retry hint; it must not silently drop durable coordination events.

20.2 Disaster recovery

Define RPO/RTO by deployment tier during production readiness. At minimum, use automated PostgreSQL backups and WAL/PITR, object-storage versioning/lifecycle protection, broker stream backup/configuration recovery, tested secret-store recovery, IaC-based environment reconstruction and periodic restore drills. Event and message retention must be consistent with legal, tenant and billing/audit obligations.

21. Implementation Repository and Coding Standards

21.1 Suggested repository layout

```
devweave/
 src/
  Web/
   DevWeave.Web.OwnerStudio/        # Next.js app (separate frontend repo also acceptable)
   DevWeave.Web.ClientPortal/
   DevWeave.Web.Admin/
  Hosts/
   DevWeave.ApiHost/
   DevWeave.McpGateway/
   DevWeave.A2aGateway/
   DevWeave.WorkerHost/
   DevWeave.RunnerGateway/
   DevWeave.ManagedRunnerHost/
  Modules/
   Identity/
   Tenancy/
   Projects/
   Knowledge/
   Planning/
   Agents/
   Coordination/
   Communication/
   Integration/
   Governance/
```

```
    Commercial/
    Observability/
  Contracts/
    DevWeave.Contracts.Domain/
    DevWeave.Contracts.Agent/
    DevWeave.Contracts.Protocol/
  Adapters/
    DevWeave.Adapter.GitHub/
    DevWeave.Adapter.GitLab/
    DevWeave.Adapter.ClaudeCode/
    DevWeave.Adapter.Codex/
    DevWeave.Adapter.OpenHands/
    DevWeave.Adapter.ManagedContainer/
tests/
  Unit/
  Integration/
  Contract/
  Security/
  EndToEnd/
  Load/
  Chaos/
deploy/
  docker/
  kubernetes-or-container-platform/
  terraform/
docs/
  adr/
  api/
  runbooks/
  threat-models/
```

21.2 Module dependency rules

- Modules expose application interfaces/contracts; one module must not write another module's tables directly.
- Domain layer has no dependency on ASP.NET Core, EF Core, MCP SDK, A2A SDK, provider SDK or cloud provider SDK.
- Protocol gateways translate external protocol messages into application commands/queries; no business rules live solely in protocol handlers.
- Adapters implement ports defined by application/domain contracts.
- Cross-module events are versioned integration contracts with explicit ownership.
- All public API schemas and protocol adapter conformance tests are version-controlled.

21.3 Coding and review standards

- Nullable reference types enabled; explicit validation and structured error handling.
- All state mutations use transactional boundaries and optimistic concurrency where relevant.
- All external calls use timeout, cancellation, retry policy and idempotency where safe.
- No secrets or sensitive payloads in source control, logs or test snapshots.
- Database migrations are reviewed and tested against production-like datasets.
- Every new event/tool/API has owner, authorization rule, schema version and tests.
- Every new agent/provider adapter has capability declaration, conformance tests, failure modes and support policy.

22. Delivery Roadmap and Team Workstreams

The sequence below is a technical delivery order, not a fixed calendar commitment. Teams may parallelize only after interface contracts and ownership boundaries are stable.

Phase	Primary deliverables	Exit gate
P0 Architecture foundation	ADRs, domain glossary, solution skeleton, identity/tenant context, schema standards, local dev topology, CI.	Tenant/project boundaries and module dependencies approved; basic security tests pass.

Phase	Primary deliverables	Exit gate
P1 Knowledge and project foundation	Project, repository connections, knowledge objects/revisions/relationships, plan models, audit/outbox, artifact storage.	Example project can be modelled; plan validates and locks immutably; history export works.
P2 External-agent coordination pilot	Agent registry, MCP gateway, A2A gateway, project messaging, event replay, task scheduler, leases, runner connector, GitHub integration.	Two distinct agent implementations collaborate on one locked campaign and safely integrate PRs.
P3 Commercial agency release	Multi-project tenant operations, agency/client views, subscriptions/entitlements, usage metering, GitLab, audit/reporting and customer onboarding.	Pilot customers can onboard, execute, measure and renew using external agents.
P4 Managed execution	Runner orchestrator, isolated workspaces, secrets broker, quotas, cleanup, billing metering and hybrid campaigns.	External and managed agents participate in one campaign under identical governance.
P5 Enterprise / intelligence	Private deployment, SSO enhancements, advanced policy, reporting, partner APIs and optimization features.	Security, reliability, support and deployment controls meet enterprise acceptance.

22.1 Recommended team ownership

Workstream	Accountable team	Key output
Domain and Knowledge	Backend / Domain team	Canonical model, schema registry, revisions, provenance, search, export.
Coordination and Messaging	Backend / Distributed systems team	Campaigns, scheduling, leases, channels, durable event delivery.
MCP and A2A	Agent interoperability team	Conformant gateways, mapping, adapter SDK, protocol test harness.
Execution and Runners	Platform / Runtime team	Customer runner, managed workspaces, isolation, secret delivery, usage.
Repository and Verification	Integration team	Git provider adapters, PR/integration queue, CI evidence.
Experience and Commercial	Frontend / Product engineering	Owner studio, agency/client portal, plans/usage surfaces.
Security and QA	Security/QA/DevOps	Threat model, tenant isolation tests, conformance, load/chaos, release readiness.

23. Verification and Acceptance Strategy

23.1 Required test layers

Test type	What must be verified
Unit	Domain invariants, state transitions, policy evaluation, lease token validation, schema validation.
Integration	PostgreSQL constraints/RLS, outbox transaction, NATS delivery, object storage, secret broker and provider adapters.
Protocol contract	MCP version/transport/tool schemas; A2A 1.0 Agent Card, request/response, task state, streaming/push capability and error mapping.
Security	Cross-tenant IDOR, project scope escalation, SSRF, forged Agent Card, replay, token leakage, prompt injection, secret/source scanning.
Concurrency	Simultaneous lease claims, heartbeat race, stale fencing token, duplicate webhook, duplicate A2A message and integration queue race.
End-to-end	Locked-plan campaign with multiple agents across local/remote/managed environments, collaboration and verified integration.
Load / soak	MCP connections, A2A tasks, message fan-out, event replay, campaign scheduler and runner fleet.
Chaos / recovery	Broker unavailable, DB failover, runner partition, agent crash, provider throttling, lost notification and cleanup failure.

23.2 Mandatory end-to-end acceptance scenarios

1. Create a tenant, project, multiple repositories and a typed knowledge model. Approve and lock a plan baseline containing dependency-linked work packages.
2. Register two different external agent frameworks on distinct machines. Both receive only project/campaign-scoped permissions and context.
3. Create an execution campaign bound to the exact locked plan hash and repository baseline commit SHA.
4. Scheduler assigns independent work packages to different agents; a dependency-bound work package remains unassigned until prerequisites satisfy policy.
5. Agent A requests specialist collaboration with Agent B through A2A Gateway. Message/task is authorized, recorded, routed and correlated to a DevWeave work item.
6. Project-wide coordination event is committed durably, delivered to live subscribers and replayed after one agent disconnects.
7. Agent A loses connection and lease expires; another eligible agent may be reassigned only after new fencing token is issued. Stale agent cannot submit accepted completion.
8. Agents submit branch/commit/PR references and evidence. No source files or diffs are persisted in DevWeave.
9. Integration queue runs required checks, identifies a conflict, routes it for resolution and blocks merge until approved gates pass.
10. Customer-managed and DevWeave-managed agents participate in a hybrid campaign; metering attributes execution usage to the correct tenant/project.
11. Investor/client reviewer sees only authorized released progress and evidence; cannot access private agent prompts or restricted architecture/security records.

23.3 Release exit criteria

- All mandatory security and tenant-isolation tests pass; no unresolved critical/high issue in release scope.
- Protocol conformance tests pass for declared MCP/A2A versions and capabilities.
- Durable message acceptance, replay, deduplication and lease fencing invariants pass under failure injection.
- Source-code and secret scanning tests pass for all supported ingestion paths.
- Operational dashboards, alerting, backup/restore and incident runbooks are complete.
- Customer onboarding, runner installation, credential revocation and tenant export have been exercised end-to-end.

24. Risks, Open Decisions and ADR Backlog

ID	Open decision	Recommended default	Owner / gate
OD-01	Broker technology and topology	NATS JetStream in pilot; validate durability, fan-out, operations and managed offering.	Platform architect before P2.
OD-02	MCP SDK and exact compatibility matrix	Use SDK supporting MCP 2026-07-28 semantics; isolate in adapter and pin version.	Agent interoperability lead before P2.
OD-03	A2A protocol binding	HTTP+JSON/JSON-RPC A2A 1.0.0 first; add gRPC only for measured need.	Agent interoperability lead before P2.
OD-04	External runner transport	Outbound HTTPS streaming/WebSocket or gRPC with fallback; select through spike and network tests.	Runtime lead before P2.
OD-05	Managed isolation level	Rootless containers for trusted workloads; evaluate microVM isolation for untrusted multi-tenant workloads.	Security + platform before P4.
OD-06	Repository provider abstraction	GitHub and GitLab first; provider-neutral capability contract.	Integration lead before P2/P3.
OD-07	Plan change during active campaign	Affected scope freezes; explicit change request; new baseline or controlled amendment.	Product Owner before P2.
OD-08	Message retention and legal hold	Configurable tenant policy with default retention; audit, billing and legal hold rules.	Security/legal before pilot.
OD-09	Direct A2A peer networking	Disabled by default; mediated gateway only; optional enterprise exception.	Security before GA.

ID	Open decision	Recommended default	Owner / gate
OD-10	Semantic search/vector store	PostgreSQL FTS first; vector index only after evaluation and permission-safe design.	Knowledge lead after P2.
OD-11	Billing provider and commercial tax treatment	Provider-neutral usage ledger; select billing integration per target market.	Product/Finance before P3.
OD-12	Service extraction threshold	Extract only for independent scaling/isolation or release cadence; ADR required.	Architecture governance continuous.

24.1 Principal risks and mitigations

Risk	Mitigation
Protocol churn or incompatible agent implementations	Pin protocol versions, capability negotiation, adapter isolation and conformance suites.
A2A Agent Card endpoint SSRF or spoofing	Trust registry, TLS, signed cards where available, URL allowlists, DNS/IP validation and egress proxy.
Cross-agent prompt injection	Treat messages as untrusted data; never promote content to policy or approval; separate agent instructions from platform commands.
Lease and repository race conditions	DB transactional lease, fencing token, isolated workspaces, protected branches and integration queue.
Project message/event overload	Per-tenant quotas, pagination, retention, backpressure, broker partitioning and monitoring.
Managed execution cost/margin risk	BYO provider keys by default, metering, quotas, budget controls and runtime provider choice.
Source-code leakage through logs or transcripts	Allowlisted metadata ingestion, scanning/redaction, no raw transcript retention by default and short-lived workspace cleanup.
Monolith complexity	Strict module contracts, dependency tests, bounded contexts and measured service extraction.

Appendix A. Initial Entity and Table Catalogue

The following is a logical catalogue. It is not a mandate to create one physical table per concept if an approved aggregate design provides the same integrity, queryability and audit properties.

Entity / table family	Key relationships and important fields
ExecutionCampaign	tenant_id, project_id, plan_baseline_id, policy_set_id, state, execution_mode, started_at.
WorkItem	plan_baseline_id, work_key, type, state, priority, required_capabilities, scope_refs, expected_version.
WorkItemDependency	predecessor_work_item_id, successor_work_item_id, dependency_type, condition, created_from_baseline.
AgentRegistration	principal_id, profile_id, tenant_id, allowed_project_ids or project grants, protocol_support, trust_tier, status.
RuntimeInstance	agent_registration_id, runner_id, provider_id, endpoint/session reference, capabilities, health, last_heartbeat.
Assignment	campaign_id, work_item_id, agent_registration_id, runtime_instance_id, state, attempt, version.
TaskLease	assignment_id, lease_token, fencing_token, issued_at, expires_at, heartbeat_at, released_at.
WorkspaceSession	assignment_id, provider, workspace_ref, branch_ref, base_commit_sha, expires_at, cleanup_state.
A2ATaskMapping	devweave_assignment_id, remote_agent_id, a2a_task_id, a2a_context_id, protocol_version, remote_state.
CoordinationMessage	channel_id, sequence, sender_id, recipient_scope, message_type, schema_version, classification, payload_ref, idempotency_key.

Entity / table family	Key relationships and important fields
CoordinationEvent	event_id, project_id, campaign_id, aggregate_type/id, event_type, aggregate_version, correlation/causation IDs.
MessageDelivery	message_id, recipient_id, state, attempt_count, next_attempt_at, delivered_at, last_error_code.
IntegrationRequest	assignment_id, repository_id, branch/PR ref, base_sha, head_sha, validation_state, review_state, merge_commit_sha.
UsageRecord	tenant_id, project_id, runtime_id, metric, quantity, unit, rate_card_version, occurred_at, billing_period.

Appendix B. Example Contracts

B.1 Agent registration (simplified)

```
{
 "agentRegistrationId": "agtreg_01J...",
 "tenantId": "ten_01J...",
 "projectId": "prj_01J...",
 "agentProfile": "backend-specialist",
 "principalId": "prn_01J...",
 "integrationMode": "native-a2a",
 "protocols": {
  "mcpClient": true,
  "a2aVersion": "1.0",
  "a2aStreaming": true,
  "a2aPush": false
 },
 "skills": ["dotnet-api", "postgresql", "integration-testing"],
 "runtimeMode": "customer-managed",
 "trustTier": "project-approved",
 "status": "admitted"
}
```

B.2 Locked plan baseline (simplified)

```
{
 "planBaselineId": "plb_01J...",
 "projectId": "prj_01J...",
 "version": 7,
 "contentHash": "sha256:...",
 "repositories": [
  { "repositoryId": "repo_1", "targetBranch": "main", "baselineCommitSha": "abc123..." }
 ],
 "workItemCount": 42,
 "approvedBy": "prn_owner_1",
 "lockedAt": "2026-11-10T09:00:00Z",
 "state": "locked"
}
```

B.3 Assignment and lease (simplified)

```
{
 "assignmentId": "asn_01J...",
 "campaignId": "cmp_01J...",
 "workItemId": "DW-125",
 "agentRegistrationId": "agtreg_01J...",
```

```
 "runtimeInstanceId": "run_01J...",
 "planBaselineId": "plb_01J...",
 "lease": {
  "leaseId": "lse_01J...",
  "fencingToken": 18,
  "expiresAt": "2026-11-10T09:15:00Z"
 },
 "workspace": {
  "repositoryId": "repo_1",
  "branch": "dw/cmp-22/DW-125/agent-backend",
  "baseCommitSha": "abc123..."
 }
}
```

B.4 A2A-to-DevWeave task mapping (simplified)

```
{
 "mappingId": "map_01J...",
 "tenantId": "ten_01J...",
 "projectId": "prj_01J...",
 "campaignId": "cmp_01J...",
 "devweaveWorkItemId": "DW-125",
 "assignmentId": "asn_01J...",
 "a2aAgentRegistrationId": "agtreg_database_1",
 "a2aProtocolVersion": "1.0",
 "a2aTaskId": "remote-task-id",
 "a2aContextId": "remote-context-id",
 "mappingState": "active"
}
```

B.5 Execution result contract (simplified)

```
{
 "assignmentId": "asn_01J...",
 "leaseId": "lse_01J...",
 "fencingToken": 18,
 "result": "submitted-for-review",
 "repositoryReferences": {
  "branch": "dw/cmp-22/DW-125/agent-backend",
  "commitSha": "def456...",
  "pullRequestUrl": "https://git.example/org/repo/pull/125"
 },
 "evidence": [
  { "kind": "ci-run", "externalId": "ci-881", "status": "passed" },
  { "kind": "test-report", "externalId": "test-441", "status": "passed" }
 ],
 "summary": "Implemented the approved endpoint contract.",
 "containsSourceCode": false,
 "submittedAt": "2026-11-10T11:00:00Z"
}
```

Contract note

Examples are illustrative, not final JSON Schema. The contract team must publish versioned schemas, validation rules, required fields, error codes, idempotency semantics and compatibility policy before implementation.

Appendix C. Event Catalogue

Event name	Producer	Principal consumers / action
PlanBaselineLocked	Plan Service	Campaign creation, audit, notifications.
ExecutionCampaignStarted	Coordination Engine	Scheduler, dashboards, agent context.
AgentRegistered / AgentRevoked	Agent Registry	Scheduler, A2A Gateway, audit.
AgentPresenceChanged	Runner/Registry	Scheduler projection, UI; ephemeral.
WorkItemReady	Planning/Coordination	Scheduler.
AssignmentOffered / Accepted	Coordination Engine	Runner, agent adapter, dashboards.
LeaseGranted / Renewed / Expired / Revoked	Coordination Engine	Runner, scheduler, audit.
A2AMessageAccepted / Routed / Failed	A2A Gateway + Communication	Delivery workers, event journal, observability.
CoordinationMessageCommitted	Project Communication	MCP subscriptions, recipient fan-out.
ExecutionStarted / Blocked / Submitted	Runtime Adapter	Coordination, knowledge, dashboards.
EvidenceSubmitted	Evidence Service	Verification, report and audit.
IntegrationRequestCreated / ChecksPassed / ChecksFailed	Integration Service	Review queue, scheduler, dashboard.
RepositoryMergeConfirmed	Repository Adapter	Work-item verification, release traceability.
ManagedUsageRecorded	Managed Runner	Metering and billing.
PlanChangeApproved	Governance	Affected work recalculation and campaign update.

Appendix D. References

[R1] Agent2Agent Protocol Specification v1.0.0 — https://a2a-protocol.org/v1.0.0/specification/
Official A2A specification covering operations, tasks, messages, Agent Cards, streaming, push notifications, versioning and security.

[R2] A2A Protocol - A2A and MCP — https://a2a-protocol.org/latest/
Official explanation of the distinct and complementary roles of MCP and A2A.

[R3] Model Context Protocol Specification 2026-07-28 — https://modelcontextprotocol.io/specification/2026-07-28
Official MCP protocol specification for the target baseline.

[R4] MCP Streamable HTTP Transport 2026-07-28 — https://github.com/modelcontextprotocol/modelcontextprotocol/blob/main/docs/specification/2026-07-28/basic/transports/streamable-http.mdx
Official HTTP transport details, endpoint requirements, request streams, subscriptions/listen and security requirements.

[R5] PostgreSQL Row Security Policies — https://www.postgresql.org/docs/current/ddl-rowsecurity.html
Database-level row filtering and write policies; apply with correct database role design.

[R6] JSON Schema 2020-12 — https://json-schema.org/draft/2020-12
Schema validation baseline for extensible knowledge payloads.

[R7] NIST SP 800-162: Guide to Attribute Based Access Control — https://csrc.nist.gov/pubs/sp/800/162/upd2/final
ABAC conceptual model for subject, object, action and environment attributes.

[R8] OpenTelemetry Documentation — https://opentelemetry.io/docs/
Cross-service tracing, metrics and logs.

[R9] Transactional Outbox Pattern — https://microservices.io/patterns/data/transactional-outbox.html
Reference pattern for reliable event publication from a database transaction.

Engineering Conclusion

DevWeave should be implemented as a knowledge-first, protocol-interoperable software delivery control plane. The core platform owns project knowledge, approved plan baselines, campaign/task authority, leases, agent registry, durable coordination events and integration governance. MCP gives agents a standard way to use DevWeave capabilities; A2A enables interoperable agent collaboration. Durable project messaging and event infrastructure preserve state and replay across disconnections. External and managed runtimes remain replaceable adapters under one common policy model.

Implementation sequencing guardrails

- Freeze and version canonical domain, event, REST, MCP and A2A contracts before expanding provider adapters.
- Establish tenant identity, project authorization, agent admission and audit controls before enabling inter-agent communication.
- Deliver durable messaging, idempotency, task leases and fencing before sophisticated autonomous scheduling.
- Prove customer-managed runners and repository integration before introducing DevWeave-managed execution at scale.
- Keep source code in customer-controlled repositories; persist only approved metadata, references, evidence and permitted non-code artifacts.

Implementation north star

One project. One approved plan baseline. Many independently executing agents. One authoritative coordination state. One or more protected code repositories. Verified and traceable delivery.